Variational Auto Encoders (VAEs) —

1. Core Idea of Variational Auto Encoders (VAE)

Definition

A Variational Auto Encoder (VAE) is a probabilistic generative model that learns:

- a compressed latent representation of data
- the probability distribution of the data
- how to generate new realistic samples from that learned distribution

Unlike a standard autoencoder:

- Standard Autoencoder → deterministic
- VAE → stochastic/probabilistic

---

2. Why Standard Autoencoders Fail for Generation

Standard Autoencoder Workflow

Input image → Encoder → Latent vector z → Decoder → Reconstructed image

The latent vector is deterministic.

Example:

- image of digit “2” → z2
- image of digit “7” → z7

Decoder learns:

- z2 → 2
- z7 → 7

Problem

The model only memorizes isolated latent points.

Between z2 and z7:

- latent space has no structure
- intermediate points produce garbage/noise

Important Concept

Standard autoencoders create:

- isolated islands of meaning
- not continuous semantic spaces

Therefore:

- reconstruction works
- generation fails

---

3. Why VAEs Introduce Randomness (Stochasticity)

Key Idea

VAEs force the latent space to become:

- smooth
- continuous
- structured

Instead of encoding:
z = single point

VAEs encode:
z ~ probability distribution

---

4. Encoder Output in VAE

The encoder outputs TWO vectors:

(a) Mean Vector (μ)

Represents:

- center of latent distribution
- most likely latent representation

(b) Variance / Standard Deviation Vector (σ)

Represents:

- uncertainty
- spread of distribution
- allowable variation around μ

Thus each latent dimension becomes:

$z \sim N(\mu, \sigma^2)$

---

5. Intuition Behind μ and σ

Analogy: Jazz Musician

Standard Autoencoder:

- classical sheet music
- every note fixed
- deterministic output

VAE:

- jazz chord chart
- structure exists
- improvisation allowed

μ = central melody
σ = amount of improvisation allowed

---

6. Two Sources of Variability in VAE

Source 1: Sampling Different z

Different latent points generate different concepts.

Example:

- one z → cat
- another z → dog

Source 2: Sampling from p(x|z)

Even with same latent point:

- decoder can generate slightly different outputs

This creates realistic variability.

---

7. Why Continuous Latent Space Matters

Because decoder is trained on distributions instead of points:

- neighboring latent points become meaningful
- concepts overlap smoothly

This enables:

- interpolation
- smooth morphing
- realistic generation

Example:

- smile → neutral → frown
  through continuous movement in latent space

---

8. Marginal Likelihood

Objective of Generative Models

We want model distribution to match real data distribution.

We maximize:

$\log p_\theta(x)$

where:

- x = observed data
- θ = decoder parameters

---

9. Marginal Likelihood Equation

$\log p_\theta(x) = \log \int p_\theta(x|z)p(z)dz$

Meaning of Each Term

$p_\theta(x|z)$

Probability of generating x from latent variable z.

p(z)

Prior distribution over latent variables.
Usually:

$p(z) = N(0, I)$

Integral over z

Sum over ALL possible latent representations.

---

10. Why Use Log Probability?

Probabilities are between 0 and 1.

Multiplying many probabilities:

- creates extremely tiny numbers
- causes numerical underflow

Using logarithms:

$\log(A \times B) = \log A + \log B$

Benefits:

- numerical stability
- easier optimization

---

11. Why Marginal Likelihood is Intractable

Main Reason

Need to integrate over ALL possible z values.

Latent space is:

- continuous
- high dimensional

Example:

- 512-dimensional latent space

Even checking 10 points per dimension:

10^{512} evaluations

This is computationally impossible.

---

12. Curse of Dimensionality

As dimensions increase:

- search space grows exponentially
- computation becomes impossible

This is why direct optimization cannot be performed.

---

13. Solution: Variational Inference

Instead of optimizing true likelihood directly:

Use approximation.

This approximation is:

ELBO = Evidence Lower Bound

---

14. ELBO Equation

ELBO = $E_{q\phi(z|x)}[\log p_\theta(x|z)] - KL(q_\phi(z|x) \,||\, p(z))$

This is the MOST IMPORTANT equation in VAEs.

---

15. Interpretation of ELBO

ELBO is a tug-of-war between:

(a) Reconstruction Quality

First term:

$E_{q\phi(z|x)}[\log p_\theta(x|z)]$

Goal:

- reconstruct input accurately
- maximize reconstruction quality

(b) Regularization

Second term:

$KL(q_\phi(z|x) \,||\, p(z))$

Goal:

- keep latent distributions close to standard normal
- maintain smooth latent space

---

16. Approximate Posterior $q_\phi(z|x)$

True posterior:

$p(z|x)$

is intractable.

So encoder learns approximation:

$q_\phi(z|x)$

This means:

- encoder predicts likely latent regions
- avoids searching entire latent universe

---

17. Reconstruction Term

Workflow:

Input x
→ Encoder predicts μ and σ
→ Sample z
→ Decoder reconstructs x
→ Measure reconstruction quality

Good reconstruction:

- increases ELBO
- improves generation quality

---

18. Reparameterization Trick

Problem

Sampling is random.

Random operations are not differentiable.

Backpropagation requires:

- deterministic computational graph

Therefore:

- gradients cannot flow through random sampling directly.

---

19. Reparameterization Equation

$z = \mu + \sigma \cdot \varepsilon$

where:

- $\varepsilon \sim N(0,1)$
- ε is external random noise

---

20. Why Reparameterization Works

Randomness is moved outside the network.

μ and σ become deterministic outputs.

Now gradients can flow through:

- μ
- σ

Thus:

- stochastic sampling becomes differentiable
- backpropagation works

---

21. KL Divergence

Definition

KL divergence measures difference between two probability distributions.

If distributions are identical:

KL = 0

Greater difference:

- larger KL penalty

---

22. Why KL Divergence is Needed

Without KL term:

- encoder collapses $\sigma \to 0$
- model becomes deterministic
- latent space becomes disconnected again

This destroys generative capability.

KL divergence prevents:

- isolated latent points
- overconfidence
- memorization

---

23. Prior Distribution p(z)

Usually chosen as:

$p(z) = N(0, I)$

Reasons:

- smooth latent space
- centered distribution
- easy sampling
- continuous geometry

---

24. ELBO as Lower Bound

Fundamental identity:

$\log p(x) = \text{ELBO} + KL(q(z|x) \,||\, p(z|x))$

Since KL divergence ≥ 0:

$\text{ELBO} \leq \log p(x)$

Thus:

- ELBO is lower bound of true likelihood.

---

25. Why Maximizing ELBO Works

If ELBO increases:

- lower bound rises
- true likelihood also rises

Therefore:

- optimizing ELBO indirectly optimizes true likelihood.

---

26. "Box and Ceiling" Analogy

Ceiling

True marginal likelihood:

log p(x)

Impossible to directly reach.

Box

ELBO.

Can be calculated.

As box rises:

- ceiling must also rise.

Reason:

- gap between them is non-negative KL divergence.

---

27. Training Phase vs Generation Phase

Training Phase

Uses:

- encoder
- decoder
- ELBO optimization

Goal:

- organize latent space
- learn distributions

Generation Phase

Encoder removed.

Steps:

1. Sample:
   $z \sim N(0, I)$
2. Decode:
   $x \sim p\theta(x|z)$

Result:

- completely new generated data

---

28. Latent Space Interpolation

Because latent space is smooth:

- interpolation becomes meaningful

Example:

- smiling face → neutral face → sad face

Moving through latent space changes concepts continuously.

---

29. Final Objective of VAE

Goal:

$p_\theta(x) \approx p_{data}(x)$

Meaning:

- model distribution approximates real data distribution.

If successful:

- generated samples become indistinguishable from real data.

---

30. Key Exam Statements (Memorize)

Statement 1

Standard autoencoders reconstruct data but fail at generation because latent space is discontinuous.

Statement 2

VAEs solve this by learning probability distributions instead of deterministic latent vectors.

Statement 3

Marginal likelihood is intractable because integration over high-dimensional continuous latent space is computationally impossible.

Statement 4

ELBO provides a tractable lower bound for optimizing the model.

Statement 5

Reparameterization trick allows stochastic sampling while preserving differentiability.

Statement 6

KL divergence regularizes latent space and prevents memorization.

Statement 7

Continuous latent space enables interpolation and realistic data generation.

---

31. Important Equations Cheat Sheet

Marginal Likelihood

$\log p_\theta(x) = \log \int p_\theta(x|z)p(z)dz$

ELBO

$ELBO = E_{q_\phi(z|x)}[\log p_\theta(x|z)] - KL(q_\phi(z|x) \,||\, p(z))$

Reparameterization Trick

$z = \mu + \sigma \cdot \varepsilon$

ELBO Identity

$\log p(x) = \text{ELBO} + KL(q(z|x)||p(z|x))$

---

32. Frequently Asked Open-Book Exam Questions

Q1. Why do VAEs use stochastic latent variables?

Answer Points:

- deterministic latent points create disconnected spaces
- interpolation fails
- VAEs use distributions
- creates smooth continuous latent geometry
- enables realistic generation

Q2. Why is marginal likelihood intractable?

Answer Points:

- integral over all latent variables
- latent space continuous and high-dimensional
- curse of dimensionality
- exponential computational explosion

Q3. Explain ELBO intuitively.

Answer Points:

- lower bound on true likelihood
- balances reconstruction and regularization
- reconstruction term improves accuracy
- KL term maintains smooth latent structure

Q4. Why is the reparameterization trick necessary?

Answer Points:

- sampling is non-differentiable
- gradients cannot flow through randomness
- rewrite sampling as deterministic transformation
- enables backpropagation

Q5. What role does KL divergence play?

Answer Points:

- compares encoder distribution with prior
- prevents overfitting/memorization
- keeps latent space smooth and continuous
- avoids collapse into deterministic points

Denoising Diffusion Probabilistic Models (DDPMs) —

1. Core Idea of Diffusion Models

Definition

Diffusion models are generative models that learn how to:

- gradually destroy data using noise
- then reverse the destruction process
- generate new realistic samples from pure noise

Core Principle:

"If destruction is gradual and mathematically controlled, the reverse process can be learned."

---

2. Basic Intuition

Imagine:

- Start with clean image x_0
- Add tiny Gaussian noise repeatedly
- Eventually image becomes pure static

Forward process:

image → noisy image → more noisy → pure Gaussian noise

Reverse process:

pure noise → structured image → clean image

---

3. Markov Chain

Diffusion models use a Markov chain.

Markov Property

Future depends only on present.

Mathematically:

$q(x_t \mid x_{t-1}, \ldots, x_0) = q(x_t \mid x_{t-1})$

Meaning:

- current noisy state contains all required information
- earlier states are unnecessary

---

4. Forward Diffusion Process

Goal

Gradually destroy data by adding Gaussian noise.

Each step:

$x_{t-1} \to x_t$

adds small random Gaussian noise.

---

5. Forward Transition Distribution

$q(x_t \mid x_{t-1}) = N(\sqrt{\alpha_t}\, x_{t-1}, (1-\alpha_t)I)$

---

6. Meaning of Terms

α_t (alpha)

Controls signal preservation.

Since $\alpha_t < 1$:

- signal shrinks slightly every step

Acts as:

- signal contraction factor

β_t (beta)

Defined as:

$\beta_t = 1 - \alpha_t$

Controls noise variance.

Acts as:

- noise injection strength

ε (epsilon)

Random Gaussian noise.

Sampled from:

$\varepsilon \sim N(0, I)$

---

7. Reparameterized Forward Equation

$x_t = \sqrt{\alpha_t}\, x_{t-1} + \sqrt{(1-\alpha_t)}\, \varepsilon$

Interpretation:

- first term preserves old signal

- second term injects fresh randomness

---

8. Noise Schedule

Noise is not constant.

β_t increases over time.

Typical schedule:

- early steps → tiny noise
- later steps → larger noise

Reason:

- avoid destroying image too quickly
- ensure gradual smooth corruption

---

9. Why Gaussian Noise is Used

Gaussian transitions provide:

(a) Markov structure

Memoryless process.

(b) Guaranteed convergence

All data eventually becomes:

$N(0, I)$

(c) Closed-form mathematics

Allows direct sampling at arbitrary time steps.

---

10. Trajectory vs Marginal Distribution

Trajectory

Probability of a SPECIFIC noisy path.

Tracks:

$x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \ldots \rightarrow x_T$

Focuses on exact journey.

Marginal Distribution

Probability of being at state x_t regardless of path.

Focuses only on:

- current noisy state
- not history

---

11. Law of Marginalization

To calculate marginal distribution:

Need to integrate over all intermediate states.

This becomes computationally enormous.

But Gaussian properties simplify the integral.

---

12. Cumulative Alpha ($\bar{\alpha}_t$)

Defined as:

$\bar{\alpha}_t = \prod \alpha_i$

(product from i=1 to t)

---

13. Physical Meaning of $\bar{\alpha}_t$

$\bar{\alpha}_t$ represents:

- total surviving signal strength
- remaining original information

$1 - \bar{\alpha}_t$ represents:

- accumulated noise variance

---

14. Closed-Form Marginal Distribution

Most important DDPM equation:

$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{(1-\bar{\alpha}_t)}\, \varepsilon$

This allows:

- direct jump to any timestep
- no need to simulate all previous steps

---

15. Importance of Closed Form

Without this equation:

- need sequential simulation
- extremely slow training

With closed form:

- sample arbitrary timestep instantly
- efficient training

---

16. Induction Proof (Exam Important)

You may be asked to prove:

$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{(1-\bar{\alpha}_t)}\, \varepsilon$

using mathematical induction.

---

17. Induction Proof Structure

Base Case

Show equation holds for $t = 1$.

Substitute:

$\bar{\alpha}_1 = \alpha_1$

Matches original forward equation.

Inductive Hypothesis

Assume formula true for:

x_{t-1}

Inductive Step

Substitute x_{t-1} into forward equation.

Use Gaussian properties:

- means add linearly
- variances add for independent Gaussians

Final expression collapses to:

$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{(1-\bar{\alpha}_t)}\, \varepsilon$

---

18. Signal-to-Noise Ratio (SNR)

Defined as:

$SNR = \bar{\alpha}_t / (1 - \bar{\alpha}_t)$

Interpretation:

- numerator = signal
- denominator = noise

As t increases:

- signal decreases
- noise dominates

Eventually:

$SNR \to 0$

---

19. Mutual Information

Measures relationship between:

- original image x_0
- noisy image x_t

As diffusion progresses:

- mutual information decreases
- original structure disappears

---

20. Reverse Diffusion Process

Goal:

Recover clean image from pure noise.

Reverse chain:

$x_T \to x_{T-1} \to \ldots \to x_0$

---

21. Why Reverse Process is Difficult

True reverse distribution:

$q(x_{t-1} \mid x_t)$

is intractable.

Reason:

- requires integrating over entire unknown data distribution
- infinitely many possible original images could produce same noisy image

---

22. Solution: Learn Reverse Process

Introduce neural network parameterized by θ.

Learn approximation:

$p_\theta(x_{t-1} \mid x_t)$

---

23. Reverse Distribution

Modeled as Gaussian:

$p_\theta(x_{t-1}|x_t) = N(\mu_\theta(x_t,t), \sigma_t^2 I)$

Usually:

- variance fixed
- network predicts only mean

---

24. Why Predict Noise Instead of Image?

Direct Image Prediction Problem

If network predicts full image directly:

- uncertainty becomes huge
- network averages possibilities
- outputs blurry images

Noise Prediction Advantage

Predicting noise:

- simpler task
- isolates random corruption
- preserves sharp structure
- produces high-quality images

This is one of the MOST IMPORTANT concepts.

---

25. DDPM Training Procedure

Step 1

Sample clean image x_0.

Step 2

Sample random timestep t.

Step 3

Sample Gaussian noise ε.

Step 4

Generate noisy image:

$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{(1-\bar{\alpha}_t)}\, \varepsilon$

Step 5

Feed:

- noisy image x_t
- timestep t
  into neural network.

Network predicts:

$\varepsilon_\theta(x_t,t)$

---

26. Loss Function

DDPM uses simple Mean Squared Error:

$L_{simple} = E[||\varepsilon - \varepsilon_\theta(x_t,t)||^2]$

Meaning:

- compare true noise vs predicted noise
- minimize prediction error

---

27. Reverse Mean Equation

The predicted noise is converted into reverse mean.

Key idea:

- subtract predicted noise

- restore signal scale

Physical interpretation:

- isolate structure from chaos

---

28. U-Net Architecture

Standard DDPM architecture:

U-Net

Reason:

- excellent for image-to-image tasks
- preserves spatial structure

---

29. Structure of U-Net

Encoder Path

Downsamples image.

Purpose:

- compress image
- learn semantic understanding
- capture global structure

Bottleneck

Lowest-resolution representation.

Network learns:

- deep abstract concepts
- object identity
- global relationships

Decoder Path

Upsamples image.

Purpose:

- reconstruct high-resolution output
- restore detail

---

30. Skip Connections

Most important architectural feature.

Skip connections:

- transfer high-resolution spatial information

- bypass bottleneck
- preserve precise locations/details

Without skip connections:

- blurry outputs
- poor spatial alignment

---

31. Time Conditioning

The network must know timestep t.

Reason:

- denoising at t=10 differs from t=900

Low noise:

- local refinement

High noise:

- global structural reconstruction

---

32. Sinusoidal Time Embeddings

Timestep integer converted into continuous vector.

Uses:

- sine functions
- cosine functions
- multiple frequencies

Same principle used in:

- transformer positional encoding

---

33. Purpose of Time Embeddings

Time embedding tells network:

- current noise regime
- how aggressively to denoise
- which features to prioritize

Acts as global modulation signal.

---

34. DDPM Sampling

Generation process:

1. Start with pure Gaussian noise
2. Predict noise
3. Subtract noise

4. Repeat for all timesteps

Eventually:

- structured realistic image emerges

---

35. Major Problem with DDPM

Generation is sequential.

Requires:

- 1000 neural network evaluations
- very slow inference

---

36. DDIM (Denoising Diffusion Implicit Models)

DDIM improves sampling speed.

MOST IMPORTANT EXAM POINT:

DDIM changes sampling process only.

Training objective and neural network remain unchanged.

---

37. DDIM Key Idea

Predict clean image estimate:

$\hat{x}_0$

from noisy image directly.

Then take large deterministic steps backward.

---

38. DDIM Parameter η (eta)

Controls stochasticity.

η = 1

Sampling remains stochastic.

Behaves similar to DDPM.

η = 0

Sampling becomes deterministic.

No random noise injected.

Allows:

- large jumps between timesteps
- much faster generation

---

39. Why DDIM is Faster

Deterministic trajectories remain stable.

Therefore:

- can skip timesteps safely
- reduce 1000 steps → ~20 steps

Massive speed improvement.

---

40. Deterministic Property of DDIM

Same initial noise:
→ same final image every time.

Useful for:

- interpolation
- reproducibility
- commercial generation

---

41. DDPM vs DDIM

Feature	DDPM	DDIM
Reverse Process	Stochastic	Deterministic/Stochastic
Noise Injection	Yes	Optional
Sampling Speed	Slow	Fast
Training Objective	Same	Same
Neural Network	Same	Same
Step Skipping	Difficult	Easy

---

42. Most Important Conceptual Insight

Diffusion models do NOT memorize images.

They learn:

- statistical structure
- frequency relationships
- geometry of data distributions

Generation = sculpting structure out of chaos.

---

43. Key Exam Statements (Memorize)

Statement 1

Diffusion models gradually corrupt data with Gaussian noise and learn to reverse the process.

Statement 2

The forward process is a Markov chain where each state depends only on the previous state.

Statement 3

The closed-form marginal allows direct sampling at arbitrary timesteps.

Statement 4

Noise prediction is superior to direct image prediction because it preserves sharp structure.

Statement 5

The U-Net combines semantic understanding with spatial precision using skip connections.

Statement 6

Sinusoidal embeddings condition the network on timestep information.

Statement 7

DDIM accelerates generation by making the reverse process deterministic.

---

44. Important Equations Cheat Sheet

Forward Transition

$q(x_t|x_{t-1}) = N(\sqrt{\alpha_t}\, x_{t-1}, (1-\alpha_t)I)$

Reparameterized Forward Step

$x_t = \sqrt{\alpha_t}\, x_{t-1} + \sqrt{(1-\alpha_t)}\, \varepsilon$

Closed-Form Marginal

$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{(1-\bar{\alpha}_t)}\, \varepsilon$

Signal-to-Noise Ratio

$SNR = \bar{\alpha}_t / (1-\bar{\alpha}_t)$

DDPM Loss Function

$L_{simple} = E[||\varepsilon - \varepsilon_\theta(x_t,t)||^2]$

---

45. Frequently Asked Open-Book Exam Questions

Q1. Why does the forward process use Gaussian noise?

Answer Points:

- enables closed-form mathematics
- ensures convergence to standard normal distribution
- preserves Markov property
- mathematically tractable

Q2. Why is the reverse process intractable?

Answer Points:

- requires averaging over entire data distribution
- many possible original images map to same noisy state
- impossible exact Bayesian inversion

Q3. Why do DDPMs predict noise instead of images?

Answer Points:

- image prediction causes averaging/blurring
- noise prediction isolates corruption
- preserves structure
- easier learning problem

Q4. Explain role of U-Net in DDPM.

Answer Points:

- encoder captures semantics
- decoder reconstructs details
- skip connections preserve spatial precision
- conditioned on timestep embeddings

Q5. Difference between DDPM and DDIM?

Answer Points:

- DDPM stochastic sampling
- DDIM deterministic sampling possible
- DDIM allows timestep skipping
- both use same trained network

Advanced Diffusion Models, Score Matching & Classifier-Free Guidance —

1. Core Problem of Generative Modeling

Goal:
Learn the probability distribution of real data.

Example:

- human faces
- handwritten digits
- images
- audio

The model should generate:

- realistic new samples
- not memorized copies

---

2. Intractable Marginal Likelihood

Problem

To compute exact probability of an image:

Need to integrate over infinitely many noise trajectories.

This becomes computationally impossible.

---

3. Meaning of "Intractable"

Intractable does NOT mean:

- difficult

It means:

- physically impossible to compute exactly
- exceeds computational limits of universe

Reason:

- image space is extremely high dimensional
- infinite possible noise paths exist

---

4. Solution: ELBO

Instead of maximizing exact likelihood:

Use a lower bound:

ELBO (Evidence Lower Bound)

---

5. Why ELBO Works

If ELBO increases:

- true likelihood also increases.

Therefore:

- optimizing ELBO indirectly optimizes data likelihood.

---

6. ELBO Decomposition

ELBO contains:

(a) Reconstruction Term

Measures reconstruction quality.

(b) Prior Matching Term

Ensures latent/noise distribution matches target prior.

(c) KL Divergence Terms

Measure difference between:

- true forward process
- learned reverse process

---

7. KL Divergence

Definition

KL divergence measures difference between probability distributions.

If distributions match perfectly:

KL = 0

---

8. Why Gaussian Structure is Important

Forward diffusion uses Gaussian distributions.

Gaussian distribution is fully defined by:

- mean
- variance

If variance is fixed:

- only means need to be matched.

---

9. Massive Simplification

Because variances are fixed:

KL divergence reduces to:

Mean Squared Error (MSE)

This is one of the MOST IMPORTANT concepts.

---

10. Key Insight

Complex variational inference objective:

reduces to:

simple noise prediction.

---

11. Noise Prediction Objective

Network predicts:

$\epsilon_\theta(x_t,t)$

Goal:
Match true injected noise ε.

Loss:

$L = E[||\varepsilon - \varepsilon_\theta||^2]$

---

12. Why Noise Prediction Works

Predicting noise is mathematically equivalent to:

- ELBO optimization
- score estimation
- learning data distribution geometry

---

13. Score Function

Defined as:

$\nabla x \log p(x)$

Read as:

gradient of log probability density.

---

14. Why Use Log Probability?

Probabilities in high dimensions become extremely tiny.

Taking logarithm:

- improves numerical stability
- converts multiplication into addition
- smooths optimization landscape

---

15. Geometric Interpretation of Score

The score is:

- a vector field
- pointing toward higher probability regions.

It acts like:

- restorative gravity.

---

16. 1D Gaussian Intuition

Imagine bell curve.

Peak center:

- highest probability.

Score vectors:

- always point uphill toward peak.

Far from center:

- stronger restoring force.

---

17. Score Magnitude

Magnitude tells:

- how far sample is from realistic data.

Large score:

- sample very unrealistic.

Small score:

- sample near data manifold.

---

18. Multi-Modal Distribution

Example:

- one peak = dogs
- another peak = cats

Score field:

- pulls samples toward nearest realistic mode.

---

19. Saddle Point

Between equal modes:

- score can become zero.

Reason:

- forces cancel.

Tiny random noise determines:

- which mode sample moves toward.

---

20. Denoising Score Identity

Very important concept.

Noise prediction:

≈ score estimation.

Meaning:

- predicting noise gives direction toward clean data.

---

21. Reverse Diffusion

Generation starts from:

pure Gaussian noise.

Then repeatedly:

- predict noise

- remove noise
- move toward realistic image.

---

22. Drift + Diffusion

Reverse update has TWO parts:

(a) Drift

Deterministic movement toward higher probability.

Uses score field.

(b) Diffusion

Injects controlled Gaussian randomness.

Maintains diversity.

---

23. Why Add Noise During Reverse Process?

Very important conceptual question.

Without added stochasticity:

- all samples collapse into same mode
- model generates identical outputs

This is:

Mode Collapse.

---

24. Purpose of Reverse Noise

Noise allows:

- exploration of probability landscape
- diversity in generation
- sampling different valid outputs.

---

25. Drift-Diffusion Tradeoff

Drift

Provides realism.

Diffusion

Provides diversity.

Together:

- realistic AND varied generation.

---

26. Guided Diffusion

Unconditional diffusion asks:

"What images are realistic?"

Guided diffusion asks:

"What realistic images satisfy a condition?"

Example:

- generate only digit 3
- generate cat
- generate red car

---

27. Bayes Rule for Guidance

Goal:

$p(x|y)$

Probability of image x given condition y.

Taking gradient in log-space:

$\nabla \log p(x|y)$

$\nabla \log p(x)$
+
$\nabla \log p(y|x)$

---

28. Interpretation of Bayes Decomposition

First Term

$\nabla \log p(x)$

Unconditional score.

Ensures realism.

Second Term

$\nabla \log p(y|x)$

Condition gradient.

Steers sample toward condition.

---

29. Geometric Interpretation

Unconditional score:

- natural gravity landscape.

Condition gradient:

- tilts landscape toward desired target.

---

30. Classifier Guidance (Approach A)

Use separate classifier network.

Classifier predicts:

- probability image belongs to target class.

Gradient from classifier:

- steers reverse diffusion.

---

31. Why Differentiate w.r.t Reverse Mean?

Reverse mean represents:

- proposed next denoised step.

Gradient asks:

"How should this proposed image move to better match condition?"

---

32. Noise-Aware Classifier

Standard classifiers fail on noisy images.

Reason:

- early diffusion steps are mostly static.

Therefore classifier must be trained on:

- noisy images
- multiple timesteps.

---

33. Training Noise-Aware Classifier

Process:

- take clean images
- add varying noise levels
- train classifier to predict correct label despite corruption.

Classifier learns:

- hidden statistical structure
- not just clean edges.

---

34. Problems with Classifier Guidance

Major disadvantages:

- requires second network
- expensive gradients
- slower inference
- unstable optimization
- engineering complexity

---

35. Classifier-Free Guidance (CFG)

Most important practical breakthrough.

Key Idea:

Use ONE diffusion model for:

- conditional generation
- unconditional generation

No external classifier needed.

---

36. CFG Training Trick

During training:

- randomly drop labels sometimes.

Example:

- 80% conditioned training
- 20% null token training.

This teaches model:

- conditional denoising
- unconditional denoising.

---

37. Null Token

Represents:

- no conditioning information.

Forces model to learn:

- generic data distribution.

---

38. CFG Inference Process

Run model TWICE:

Conditional Pass

Input:

- noisy image
- condition

Output:

- ε_{cond}

Unconditional Pass

Input:

- noisy image

- null token

Output:

- ε_{uncond}

---

39. CFG Formula

ε_{CFG}

$(1+w)\ \varepsilon_{cond}$
−
$w\ \varepsilon_{uncond}$

Equivalent form:

ε_{CFG}

ε_{cond}
+
$w(\varepsilon_{cond} - \varepsilon_{uncond})$

---

40. Most Important CFG Interpretation

Difference term:

$(\varepsilon_{cond} - \varepsilon_{uncond})$

isolates:

pure conditioning direction.

Example:

- essence of “threeness”
- minus generic “numberness”.

---

41. Role of Guidance Weight w

Controls strength of conditioning.

Low w

Weak guidance.

Results:

- high diversity
- lower condition accuracy.

High w

Strong guidance.

Results:

- better condition adherence
- lower diversity

- risk of artifacts.

---

42. Too Large Guidance Weight

Very high w causes:

- oversaturation
- unrealistic textures
- “deep fried” artifacts
- manifold distortion.

Reason:

- vector field becomes over-amplified.

---

43. Diversity vs Fidelity Tradeoff

Low guidance:

- diverse outputs
- weaker prompt matching.

High guidance:

- accurate prompts
- repetitive outputs.

Need balanced sweet spot.

---

44. FID (Fréchet Inception Distance)

Gold standard metric for generative quality.

Measures:

- distance between real and generated feature distributions.

---

45. Why Raw Pixels Cannot Be Compared

Pixel-wise comparison is misleading.

Example:

- same image shifted by 1 pixel
- humans see same object
- pixel difference becomes huge.

Need semantic comparison.

---

46. Feature Space

Use pretrained InceptionV3 network.

Extract:

- deep feature activations.

Features capture:

- curves
- textures
- semantic structure

---

47. FID Procedure

Step 1

Extract feature vectors from real images.

Step 2

Compute:

- feature mean
- covariance matrix.

Step 3

Repeat for generated images.

Step 4

Compute Fréchet distance between distributions.

---

48. Meaning of FID

Lower FID:

- generated distribution closer to real distribution.

Better generation quality.

---

49. Negative FID Scores

Negative FID is NOT mathematically meaningful.

Reason:

- numerical instability
- floating-point precision errors
- covariance square-root approximations.

---

50. Why Negative FID Indicates Excellent Performance

When generated distribution:
≈ real distribution

True FID:
≈ 0.

Tiny rounding errors may push result slightly below zero.

Thus:

- negative FID indicates near-perfect overlap.

---

51. Why CFG Outperformed Classifier Guidance

CFG advantages:

- single network
- faster inference
- stable training
- no backward gradients during sampling
- better conditioning quality.

---

52. Core Conceptual Journey

Advanced diffusion models:

1. Learn score field
2. Predict noise
3. Walk probability landscape backward
4. Guide trajectories with conditions
5. Generate structured data from chaos.

---

53. Important Equations Cheat Sheet

ELBO Objective

Optimize lower bound instead of true likelihood.

Noise Prediction Loss

$L = E[||\varepsilon - \varepsilon\_\theta||^2]$

Score Function

$\nabla x \log p(x)$

Bayes Guidance

$\nabla \log p(x|y)$

$\nabla \log p(x)$
+
$\nabla \log p(y|x)$

CFG Formula

ε_CFG

ε_cond
+
w(ε_cond − ε_uncond)

---

54. Frequently Asked Open-Book Exam Questions

Q1. Why does ELBO reduce to simple MSE loss?

Answer Points:

- Gaussian distributions fully defined by mean and variance
- variance fixed
- only means differ
- KL divergence simplifies to mean squared error.

Q2. What is the score function geometrically?

Answer Points:

- vector field
- points toward high probability regions
- acts like restorative gravity
- guides denoising process.

Q3. Why is noise added during reverse diffusion?

Answer Points:

- prevents mode collapse
- preserves diversity
- enables exploration of data distribution.

Q4. Difference between classifier guidance and CFG?

Answer Points:

- classifier guidance uses external classifier
- CFG uses same diffusion model twice
- CFG simpler, faster, more stable.

Q5. Explain CFG intuitively.

Answer Points:

- subtract unconditional prediction from conditional prediction
- isolates pure conditioning direction
- amplify condition using guidance weight.

Q6. Why can FID become negative?

Answer Points:

- numerical precision issue
- finite sample estimation
- covariance matrix square-root instability
- indicates near-perfect overlap.

Flow Matching —

1. Core Idea of Flow Matching

Definition

Flow Matching is a generative modeling framework that learns:

- a continuous velocity field
- transporting probability mass
- from random noise to real data.

Instead of learning probability densities directly:

- it learns motion.

---

2. Main Goal

Transform:

$P_0 \rightarrow P_1$

where:

P_0

Base distribution.

Usually:

- standard Gaussian noise.

P_1

Target distribution.

Represents:

- real images
- audio
- structured data.

---

3. Balloon Analogy

Imagine:

- millions of balloons randomly scattered.

Goal:

- use wind field to move balloons into shape of cat/image.

The learned wind field:

- is the neural network.

---

4. Ordinary Differential Equation (ODE)

Core sampling equation:

$dx/dt = v\theta(x,t)$

MOST IMPORTANT equation in Flow Matching.

---

5. Meaning of Terms

x

Current state / particle position.

Represents:

- image coordinates in high-dimensional space.

---

t

Normalized time.

Range:

$0 \le t \le 1$

---

vθ(x,t)

Time-dependent velocity field.

Represents:

- learned wind direction and speed.

Predicted by:

- neural network with parameters θ.

---

6. Velocity Field

Velocity is a VECTOR.

Contains:

(a) Direction

Where particle should move.

(b) Magnitude

How fast particle should move.

---

7. Physical Interpretation

The neural network acts like:

- a weather system
- predicting wind at every location and time.

---

8. Why ODE Cannot Be Solved Analytically

Velocity field from neural network is:

- highly nonlinear
- curved
- time-varying.

Therefore:

- no closed-form solution exists.

Need numerical integration.

---

9. Numerical Integration

Numerical integration approximates:

- continuous trajectory
- using discrete steps.

---

10. Euler Method

Simplest ODE solver.

Equation:

$$x(t+\Delta t) \approx x(t) + \Delta t \cdot v_\theta(x,t)$$

---

11. Euler Intuition

Process:

- check current wind
- take tiny straight step
- repeat.

Approximates curved path using:

- many small linear segments.

---

12. Euler Sampling Algorithm

Step 1

Sample random noise:

$x_0 \sim P_0$

Step 2

Loop from:

$t=0 \rightarrow t=1$

Step 3

Predict velocity:

$v\theta(x,t)$

Step 4

Update state:

$x \leftarrow x + \Delta t \cdot v\theta(x,t)$

Step 5

Final x becomes generated sample.

---

13. Problem with Euler Method

Euler assumes:

- velocity constant during timestep.

If vector field curves sharply:

- trajectory overshoots.

This causes:

Integration Error.

---

14. Integration Error

Large Δt:

- inaccurate trajectories
- drift away from data manifold.

Small Δt:

- accurate but very slow.

---

15. Higher-Order Solvers

More advanced ODE solvers:

- RK2
- RK4
- Heun method.

---

16. RK4 Intuition

RK4:

- checks wind multiple times inside timestep
- anticipates curve direction
- blends estimates.

Produces:

- highly accurate trajectories.

---

17. Why RK4 is Better

Advantages:

- larger timesteps possible
- fewer integration steps
- less accumulated error
- faster generation.

---

18. Continuity Equation

Most intimidating PDE in slides.

Equation:

∂p_t(x)/∂t
+
∇ · (p_t(x)v(x,t))

0

---

19. Core Meaning of Continuity Equation

Represents:

Conservation of probability mass.

Probability cannot:

- disappear
- appear from nowhere.

---

20. First Term Interpretation

∂p_t(x)/∂t

Measures:

- local density change over time.

Example:

- region becoming crowded or empty.

---

21. Probability Flux

Term:

$p_t(x)v(x,t)$

Represents:

- amount of probability mass moving through space.

---

22. Divergence Operator

$\nabla \cdot$

Measures:

- net inflow/outflow of probability mass.

---

23. Divergence Intuition

Positive Divergence

Particles spreading outward.

Density decreases.

Negative Divergence

Particles converging inward.

Density increases.

---

24. Why Continuity Equation Matters

ODE controls:

- movement of one particle.

Continuity equation guarantees:

- entire distribution evolves correctly.

---

25. Key Insight

If velocity field is correct:

- individual particles move correctly
- whole distribution morphs correctly.

---

26. Training Problem

Need to teach neural network:

- correct velocity field.

Requires:

- target velocity.

27. Linear Interpolation Bridge

Training creates synthetic paths.

Equation:

x_t

$(1-t)x_0$
+
tx_1

28. Meaning of Variables

x_0

Noise sample.

x_1

Real data sample.

x_t

Intermediate point on straight path.

29. Why Linear Interpolation is Used

Simplest path between two points:

- straight line.

Creates deterministic training trajectories.

30. Target Velocity

Differentiate interpolation equation:

u_t

$x_1 - x_0$

MOST IMPORTANT derivation.

31. Key Insight About Velocity

Velocity becomes:

- constant along trajectory.

Reason:

- straight-line path has constant slope.

32. Flow Matching Loss Function

Loss:

$E[||v\theta(x\_t,t) - u\_t||^2]$

33. Interpretation of Loss

Train network to:

- predict true transport velocity.

Simple supervised regression.

---

34. Training Loop

Step 1

Sample:

- x_0 from noise
- x_1 from data.

Step 2

Sample random time t.

Step 3

Compute x_t using interpolation.

Step 4

Compute target velocity:

$u\_t = x_1 - x_0$

Step 5

Predict velocity:

vθ(x_t,t)

Step 6

Compute MSE loss.

---

35. Important Conceptual Question

"If x_1 already exists in dataset, why train model?"

Critical answer:

Model does NOT memorize paths.

It learns:

- average smooth transport dynamics
- generalized vector field.

---

36. Crossing Paths Problem

Random pairings create:

- intersecting trajectories
- contradictory velocity signals.

Network must average conflicting directions.

---

37. Generalization Mechanism

Because millions of paths overlap:

- network learns smooth global transport field
- not individual examples.

---

38. Difference Between Training and Sampling

Training

Uses known:

- x_0
- x_1

Constructs synthetic trajectories.

Sampling

Only noise x_0 exists.

Network uses learned velocity field to:

- generate entirely new samples.

---

39. Comparison with VAEs

VAE Problems

- optimize ELBO approximation
- posterior collapse
- blurry outputs
- probabilistic estimation complexity.

---

40. Comparison with Diffusion Models

Diffusion models:

- learn score function
- rely on stochastic differential equations.

Flow Matching:

- deterministic ODEs
- direct velocity regression.

---

41. Main Advantage of Flow Matching

Avoids:

- score estimation
- ELBO optimization
- stochastic differential equations.

Instead learns:

- deterministic transport directly.

---

42. Deterministic vs Stochastic Paths

Diffusion:

- noisy random trajectories.

Flow Matching:

- smooth deterministic flows.

This improves:

- efficiency
- stability.

---

43. Latent Flow Matching (LFM)

Instead of operating in pixel space:

- operate in latent space.

---

44. Why Latent Space is Better

Pixel space:

- huge dimensionality
- redundant information.

Latent space:

- compressed
- meaningful
- smoother.

---

45. Encoder and Decoder

Encoder

Compresses image:

$z = E\psi(x)$

Decoder

Reconstructs image from latent code.

---

46. Flow Matching in Latent Space

Replace:

$x \to z$

All equations remain identical.

---

47. Benefits of Latent Flow Matching

(a) Faster Training

Lower dimensionality.

(b) Faster Sampling

ODE integration cheaper.

(c) Smoother Geometry

Latent manifold easier to transport.

(d) Larger ODE Steps Possible

Less integration error.

---

48. Optimal Transport (OT)

Solves:

Crossing trajectory problem.

---

49. OT Core Idea

Instead of random pairing:

- pair noise points with nearest data points.

Goal:

- minimize total transport cost.

---

50. Cost Function

Usually:

Squared Euclidean distance.

---

51. Benefits of Optimal Transport

Produces:

- short trajectories
- fewer crossings
- smooth vector fields

- easier learning.

---

52. Hungarian Algorithm

Exact optimal matching algorithm.

Builds:

- cost matrix between points.

Finds:

- globally optimal assignment.

---

53. Problem with Hungarian Algorithm

Time complexity:

$O(N^3)$

Very expensive for large batches.

---

54. Sinkhorn Algorithm

Approximate OT algorithm.

Adds:

- entropy regularization.

Uses:

- efficient matrix operations.

---

55. Why Sinkhorn is Preferred

Advantages:

- GPU-friendly
- scalable
- nearly optimal.

---

56. Conditioning in Flow Matching

Goal:

- generate specific class.

Example:

- dog
- cat
- landscape.

---

57. Classifier-Free Guidance (CFG)

Industry standard conditioning method.

---

58. Core CFG Training Trick

Randomly drop labels during training.

Example:

- 90% conditioned
- 10% unconditional.

---

59. Result of Label Dropping

Network learns BOTH:

Conditional velocity

v_cond

Unconditional velocity

v_uncond

---

60. CFG Equation

v_CFG

v_uncond
+
s(v_cond − v_uncond)

---

61. Simplified CFG Equation

v_CFG

(1+s)v_cond
−
s v_uncond

---

62. Interpretation of Difference Vector

(v_cond − v_uncond)

Represents:

- pure semantic direction.

Example:

- essence of “dogness”.

---

63. Guidance Scale s

Controls:

- conditioning strength.

Small s

- realistic
- weak prompt adherence.

Large s

- strong prompt adherence
- risk of artifacts.

---

64. Oversaturation Problem

Very high guidance scale causes:

- unrealistic textures
- hyper-saturation
- distorted images.

Reason:

- semantic vector overwhelms realism.

---

65. Audio Equalizer Analogy

Unconditional Velocity

Bass/drums.

Provides:

- stable structure.

Conditional Difference

Treble/vocals.

Provides:

- semantic detail.

Guidance Scale

Treble volume knob.

Too high:

- distortion.

---

66. Why CFG is Better Than External Classifiers

Advantages:

- no second network
- lower memory usage
- faster inference

- simpler implementation.

---

67. Advanced Frontier: Consistency Models

New paradigm.

Instead of:

- simulating trajectory step-by-step.

Learn direct mapping:

$\hat{x}_0 = f\theta(x\_t,t)$

---

68. Key Idea of Consistency Models

Any point on trajectory:
→ maps directly to same final clean image.

---

69. Why Consistency Models Matter

Potentially enables:

- one-step generation
- no ODE integration.

Massive speedup.

---

70. Most Important Conceptual Insight

Flow Matching does NOT estimate probability densities directly.

It learns:

- continuous transport dynamics.

---

71. Important Equations Cheat Sheet

ODE

$dx/dt = v\theta(x,t)$

Euler Update

$x(t+\Delta t)$
$\approx$
$x(t)$
$+$
$\Delta t \cdot v\theta(x,t)$

Continuity Equation

$\partial p\_t(x)/\partial t$
$+$
$\nabla \cdot (p\_t(x)v(x,t))$

0

Linear Interpolation

x_t

$(1-t)x_0$
+
tx_1

Target Velocity

$u\_t = x_1 - x_0$

Flow Matching Loss

$E[||v\theta(x\_t,t) - u\_t||^2]$

CFG Equation

v_CFG

v_uncond
+
s(v_cond − v_uncond)

72. Frequently Asked Open-Book Exam Questions

Q1. Why does Flow Matching use ODEs?

Answer Points:

- models continuous transport
- deterministic trajectories
- avoids stochastic score estimation.

Q2. Why is Euler method inaccurate?

Answer Points:

- assumes constant velocity
- ignores curvature
- accumulates integration error.

Q3. Explain continuity equation intuitively.

Answer Points:

- conservation of probability mass
- density changes caused by inflow/outflow.

Q4. Why does Flow Matching generalize instead of memorize?

Answer Points:

- many crossing trajectories
- network averages transport directions
- learns smooth global flow field.

Q5. Difference between Flow Matching and Diffusion?

Answer Points:

- Flow Matching uses deterministic ODEs
- Diffusion uses stochastic differential equations
- Flow Matching avoids score estimation.

Q6. Why use latent flow matching?

Answer Points:

- dimensional reduction
- smoother latent manifold
- faster integration.

Q7. Why use optimal transport?

Answer Points:

- reduces trajectory crossings
- smoother vector fields
- easier learning.

Q8. Explain CFG intuitively.

Answer Points:

- amplify semantic direction
- combine conditional and unconditional velocities
- control realism vs prompt strength.

Transformers —

1. Core Idea of Transformers

Transformers are neural network architectures designed to:

- process sequences in parallel
- model long-range dependencies efficiently
- overcome limitations of RNNs and LSTMs.

Key innovation:

Self-Attention.

---

2. Autoregressive Language Modeling

Transformers for language modeling are often:

Autoregressive models.

Goal:
Predict next token given previous tokens.

---

3. Probability Factorization

Probability of sequence:

$P(x_1, x_2, \ldots, x\_T)$

is factorized as:

$P(x_1)$
$P(x_2|x_1)$
$P(x_3|x_1,x_2)\ldots$

---

4. Main Objective

At every timestep:

Predict next token probability distribution.

---

5. Training vs Sampling

Critical exam topic.

Training

Uses:

Teacher Forcing.

Model receives:

- true previous tokens.

Sampling / Generation

Model conditions on:

- its own generated outputs.

Errors can compound.

---

6. Teacher Forcing Intuition

Teacher forcing keeps model:

- on correct trajectory during training.

Equivalent to:

- giving correct previous answer while learning.

---

7. Markov Autoregressive Example

Simple binary sequence:

0 1 0 1 0 1

Model learns:

- if current token is 0 → next likely 1
- if current token is 1 → next likely 0.

---

8. Sigmoid Function

Maps real numbers into:

[0,1]

Used for:

- probability estimation.

---

9. Recurrent Neural Networks (RNNs)

RNNs introduced:

Hidden state h_t.

Purpose:

- maintain running summary of previous sequence.

---

10. RNN Recurrence

At each timestep:

- combine current input
- with previous hidden state.

Produces:

- updated hidden representation.

---

11. Backpropagation Through Time (BPTT)

RNNs trained using:

Backpropagation Through Time.

Gradients flow backward through sequence.

---

12. Vanishing Gradient Problem

Repeated multiplication by small weights:
→ gradients shrink exponentially.

Result:

- early information forgotten.
- network stops learning long dependencies.

---

13. Exploding Gradient Problem

Repeated multiplication by large weights:
→ gradients explode.

Result:

- unstable optimization.

---

14. Long Short-Term Memory (LSTM)

LSTM introduced:

Cell state c_t.

Purpose:

- preserve long-term information.

---

15. Cell State Intuition

Acts as:

- information highway.

Allows gradients to flow:

- with minimal modification.

---

16. LSTM Gates

Forget Gate

Removes irrelevant information.

Input Gate

Adds new information.

Output Gate

Selects visible information.

---

17. Role of U-Matrices

U-matrices process:

previous hidden state.

Purpose:

- context-aware gating.
- memory-dependent decisions.

---

18. Why LSTMs Still Failed

Two major bottlenecks:

(a) Sequential Computation

Cannot parallelize across sequence.

(b) Long Path Length

Information must travel through many steps.

Complexity:

$O(t-i)$

---

19. GPU Problem

GPUs prefer:

- massive parallel computation.

LSTMs force:

- step-by-step processing.

Training becomes slow.

---

20. Transformer Revolution

Transformers eliminated:

Sequential recurrence.

Replaced it with:

Self-Attention.

---

21. Main Transformer Goal

Compute:

Contextualized token representations.

Meaning of token changes depending on context.

---

22. Example: “Bank”

River bank

Bank → geography meaning.

Bank account

Bank → financial meaning.

Transformer dynamically adjusts representation.

---

23. Self-Attention

Every token can attend directly to:

- every other token.

No sequential bottleneck.

---

24. Parallel Processing

All tokens processed simultaneously.

Major reason transformers scale efficiently on GPUs.

---

25. Path Length Improvement

RNN/LSTM:

$O(t-i)$

Transformer:

$O(1)$

Any token connects directly to any other token.

---

26. Positional Encoding Problem

Self-attention alone ignores word order.

Reason:

- attention is permutation invariant.

---

27. Positional Embeddings

Solution:

Add position vectors to token embeddings.

Token representation becomes:

- meaning
- plus location.

---

28. Types of Positional Encoding

Learned embeddings

Optimized during training.

Sinusoidal embeddings

Fixed sine/cosine functions.

---

29. Query-Key-Value (QKV) Framework

Most important transformer concept.

Every token produces:

Query vector (Q)

What information am I looking for?

Key vector (K)

What information do I contain?

Value vector (V)

Actual information payload.

---

30. Attention Intuition

Attention acts like:

- soft search-and-retrieve system.

Query searches over keys.

Relevant values are extracted.

---

31. Library Analogy

Query

Search term.

Key

Folder label.

Value

Document contents.

---

32. Dot Product Attention

Similarity between Q and K measured using:

Dot Product.

Large dot product:

- high relevance.

Small/negative dot product:

- low relevance.

---

33. Attention Formula

Attention(Q,K,V)

softmax(QK^T / √d_k)V

Core transformer equation.

---

34. Why Divide by √d_k?

MOST IMPORTANT EXAM QUESTION.

Without scaling:

- dot products explode with dimension.
- softmax saturates.
- gradients vanish.

Scaling stabilizes variance.

---

35. Softmax Saturation

Large logits cause:

- winner-take-all probabilities.
- near-zero gradients.

Network stops learning.

---

36. Scaling Factor Intuition

$\sqrt{d_k}$ acts as:

- volume control knob.

Keeps logits in stable range.

---

37. Attention Weights

Softmax converts similarity scores into:

Probability weights α.

Weights sum to 1.

---

38. Final Attention Output

Weighted sum of value vectors.

Produces:

- contextualized token representation.

---

39. Multi-Head Attention

Single attention head:

- captures one type of relationship.

Multi-head attention:

- captures multiple relationships simultaneously.

---

40. Different Attention Heads

Possible head specializations:

- syntax
- long-range dependencies
- entities
- grammar
- semantics.

---

41. Multi-Head Process

Step 1

Project inputs into multiple Q,K,V spaces.

Step 2

Run attention independently per head.

Step 3

Concatenate outputs.

Step 4

Project back using W_O.

---

42. Causal Masking

Used in autoregressive transformers.

Purpose:

- prevent future token leakage.

---

43. How Mask Works

Future token scores set to:

$-\infty$

After softmax:

- probability becomes zero.

---

44. GPT vs BERT

GPT

Decoder-only.

Uses:

- causal masking.
- left-to-right generation.

BERT

Encoder-only.

Uses:

- bidirectional attention.
- full sentence context.

No causal masking.

---

45. Transformer Block Structure

Full transformer block:

1. Self-attention
2. Residual connection
3. Layer normalization
4. Feedforward network (FFN)
5. Residual connection
6. Layer normalization.

---

46. Residual Connections

Equation:

x + y

Purpose:

- preserve information
- improve gradient flow.

---

47. Why Residuals Matter

Provide:

- direct gradient highway.
- prevent vanishing gradients.

---

48. Layer Normalization

Normalizes hidden activations.

Purpose:

- stabilize training.
- prevent activation drift.

---

49. LayerNorm Formula

Subtract mean.

Divide by standard deviation.

Then apply:

- learned scale γ
- learned shift β.

---

50. Why Learn γ and β?

Strict normalization may destroy useful scaling.

γ and β allow:

- flexible rescaling.
- preserve expressivity.

---

51. Feedforward Network (FFN)

Applied independently to each token.

Purpose:

- compute nonlinear semantic transformations.

---

52. Attention vs FFN

Attention

Mixes information.

Acts like:

- librarian.

FFN

Processes information.

Acts like:

- scholar/reasoner.

---

53. FFN Architecture

Usually:

- expand hidden dimension
- apply activation
- project back.

---

54. Activation Functions

ReLU

Hard threshold.

Outputs zero for negatives.

GELU

Smooth probabilistic activation.

Used in modern transformers.

---

55. Why GELU is Better

Provides:

- smoother gradients
- fewer dead neurons
- stable optimization.

---

56. Dead Neuron Problem

ReLU may output:

0

with zero gradient.

Neuron stops learning.

---

57. Transformer Training Objective

Goal:

Minimize negative log likelihood.

Equivalent to:

- maximize probability of true next token.

---

58. Shifted Sequence Training

Input:

x_1 … x_{t−1}

Target:

x_2 … x_t

---

59. LM Head

Final linear projection layer.

Maps hidden states to:

- vocabulary logits.

---

60. Cross Entropy Loss

Measures:

- mismatch between predicted and true token distributions.

Low loss:

- correct token assigned high probability.

---

61. Backpropagation in Transformers

Gradients update:

- token embeddings
- positional embeddings
- Q,K,V matrices
- FFN weights
- LayerNorm parameters
- LM head.

---

62. Why Transformers Scale So Well

Advantages:

- parallel processing
- short information paths
- stable gradients
- GPU efficiency.

---

63. Main Limitation of Transformers

Self-attention complexity:

$O(T^2)$

Reason:

- every token attends to every other token.

---

64. Quadratic Scaling Problem

Doubling context length:
→ quadruples compute cost.

Limits very long context windows.

---

65. Emerging Alternatives

State-space models (e.g. Mamba):

Attempt to combine:

- transformer quality
- RNN linear efficiency.

---

66. Most Important Conceptual Insight

Transformers replaced sequential memory with:

Global parallel attention.

---

67. Important Equations Cheat Sheet

Autoregressive Factorization

P(x_1,...,x_T)

∏ P(x_t|x_{<t})

Attention Formula

softmax(QK^T / √d_k)V

Residual Connection

x + y

Cross Entropy Loss

−log P(true token)

---

68. Frequently Asked Open-Book Exam Questions

Q1. Why did RNNs/LSTMs fail at scale?

Answer Points:

- sequential bottleneck
- poor GPU utilization
- long-range dependency degradation.

Q2. Why is self-attention superior?

Answer Points:

- direct token connections
- O(1) path length
- parallel computation.

Q3. Explain QKV intuitively.

Answer Points:

- Query asks for information
- Key advertises information

- Value provides information.

Q4. Why divide by √d_k?

Answer Points:

- stabilize variance
- prevent softmax saturation
- preserve gradients.

Q5. Why use multi-head attention?

Answer Points:

- capture multiple relationship types simultaneously.

Q6. Difference between GPT and BERT masking?

Answer Points:

- GPT uses causal masking
- BERT uses bidirectional attention.

Q7. Difference between attention and FFN?

Answer Points:

- attention mixes information
- FFN computes nonlinear reasoning.

Q8. Why use residual connections?

Answer Points:

- preserve gradients
- stabilize deep training.

Vision Transformers (ViT) —

1. Core Idea of Vision Transformers (ViT)

Vision Transformers treat images as:

Sequences of tokens.

Main idea:

- split image into patches
- embed patches as vectors
- process them using transformer architecture.

---

2. Historical Context

Transformers originated in:

Natural Language Processing (NLP).

Key paper:

"Attention Is All You Need" (2017)

---

3. Why Transformers Revolutionized NLP

Transformers replaced:

- sequential recurrence

with:

- parallel self-attention.

Benefits:

- direct long-range interactions
- parallel GPU computation
- scalable architectures.

---

4. Vision Transformer Breakthrough

ViT paper (2021):

"An Image is Worth 16x16 Words"

Key insight:

- images can be tokenized like language.

---

5. Four Major Pillars of ViT

(a) Token-Based View

Images become sequences of tokens.

(b) Global Context

Every patch attends globally.

(c) Scaling

Performance improves predictably with model size.

(d) Multimodal Compatibility

Images and text share same token-based architecture.

---

6. CNN vs ViT Intuition

CNN

Processes:

- local neighborhoods.

Like:

- scanning image with moving magnifying glass.

ViT

Processes:

- global relationships immediately.

Every patch can directly interact with every other patch.

---

7. Why Global Attention Matters

Useful for:

- symmetry
- long-range structure
- object relationships
- generative modeling.

---

8. Image Representation Problem

Self-attention scales quadratically.

Cannot treat:

- every pixel as token.

Too expensive.

---

9. Patch Embedding

Solution:

- divide image into patches.

Each patch becomes:

- one visual token.

---

10. CIFAR-10 Example

Input image shape:

[B,C,H,W]

For one image:

[1,3,32,32]

---

11. Patch Size

Patch size:

P = 4

Meaning:

- 4×4 patches.

---

12. Number of Patches

Formula:

N = (H/P) × (W/P)

For 32×32 image:

N = 8 × 8 = 64

---

13. ConV2D Trick

Patch extraction implemented using:

ConV2D layer.

---

14. Critical ConV2D Parameters

Kernel Size = Patch Size

Stride = Patch Size

This creates:

- non-overlapping patches.

---

15. Why This is Clever

Instead of sliding convolution:

- filter jumps patch-by-patch.

Efficient patch extraction.

---

16. Embedding Dimension

Each patch projected into:

D-dimensional vector.

Example:

D = 192

---

17. Tensor Shape After Convolution

Output shape:

[B,D,H/P,W/P]

Example:

[1,192,8,8]

---

18. Flattening Spatial Grid

Flatten 8×8 grid into:

64 patch sequence.

Tensor becomes:

[1,192,64]

---

19. Final ViT Input Shape

Transpose dimensions:

[B,N,D]

Example:

[1,64,192]

MOST IMPORTANT tensor shape.

---

20. Why Positional Encoding is Needed

Transformers are:

Permutation invariant.

Without position information:

- shuffled patches look identical.

---

21. Positional Encoding Goal

Inject spatial geometry into token sequence.

---

22. Learned 1D Absolute Encoding

Add learnable vector:

P

to patch embeddings.

Equation:

X' = X + P

---

23. Intuition of Learned Encoding

Model learns:

- what top-left patches usually look like
- what bottom-right patches usually look like.

---

24. Problem with 1D Absolute Encoding

Treats image as:

- flat sequence.

Loses explicit 2D geometry.

---

25. 2D Absolute Positional Encoding

Separate embeddings for:

- x-coordinate
- y-coordinate.

Preserves:

- horizontal
- vertical relationships.

---

26. Relative Positional Encoding

Position represented relative to:

- neighboring tokens.

Focuses on:

- spatial relationships.

---

27. Relative Position Bias

Bias term added directly inside:

Attention scores.

---

28. Why Relative Encoding Helps

Important for:

- object detection
- segmentation
- dense prediction tasks.

---

29. Conditional Positional Encoding (CPE)

Positional information generated dynamically.

Adapts to:

- input image structure.

---

30. Rotary Positional Encoding (RoPE)

Uses:

- vector rotations.

Applied to:

- query and key vectors.

---

31. Why RoPE is Important

Supports:

- resolution extrapolation
- long-context generalization.

---

32. RoPE vs Absolute Encoding

Absolute Encoding

Memorizes fixed positions.

Fails on unseen resolutions.

RoPE

Encodes relative angular relationships.

Generalizes better.

---

33. CLS Token

Special learnable token prepended to sequence.

Contains:

- no image pixels.

Purpose:

- aggregate global information.

---

34. Sequence Length After CLS Token

Original:

N patches.

After CLS token:

N + 1 tokens.

Example:

65 tokens.

---

35. Why CLS Token Works

Self-attention is:

- bidirectional
- unmasked.

CLS token attends to:

- every patch.

Gradually becomes:

- global summary representation.

---

36. Backpropagation and CLS Token

Loss gradient flows back into:

- CLS token parameters.

Over training:

- token learns how to summarize images.

---

37. Alternative to CLS Token

Global average pooling.

Average all patch vectors.

Both approaches are valid.

---

38. Transformer Block Structure

Each block contains:

1. LayerNorm
2. Multi-head self-attention
3. Residual connection
4. LayerNorm
5. Feedforward network (MLP)
6. Residual connection.

---

39. Residual Connections

Equation:

$x + y$

Purpose:

- preserve information
- stabilize gradients.

---

40. Layer Normalization

Prevents:

- exploding activations
- unstable distributions.

Normalizes token representations.

---

41. Multi-Head Self-Attention

Core ViT mechanism.

Every token produces:

- Query
- Key
- Value vectors.

---

42. Query-Key-Value Intuition

Query

What am I searching for?

Key

What information do I contain?

Value

Actual contextual payload.

---

43. Dot Product Attention

Similarity measured using:

Dot Product.

High dot product:

- strong alignment.

---

44. Attention Equation

Attention(Q,K,V)

softmax(QK^T / √d_h)V

MOST IMPORTANT equation.

---

45. Why Divide by √d_h?

MOST IMPORTANT EXAM QUESTION.

Without scaling:

- dot products become huge
- softmax saturates
- gradients vanish.

---

46. Softmax Saturation

Large logits cause:

- near one-hot distributions

- flat gradients.

Learning stops.

---

47. Scaling Factor Intuition

$\sqrt{d_h}$ acts as:

- variance dampener.
- numerical stabilizer.

Keeps gradients healthy.

---

48. Attention Output

Attention weights multiply:

- value vectors.

Produces:

- contextualized token representations.

---

49. Multi-Head Attention

Different heads learn:

- different visual relationships.

Example:

- edges
- textures
- symmetry
- long-range geometry.

---

50. Why Multi-Head Attention Helps

Allows simultaneous modeling of:

- local and global patterns.

---

51. Feedforward Network (FFN)

Processes each token independently.

Purpose:

- nonlinear semantic transformation.

---

52. Attention vs FFN

Attention

Moves information.

FFN

Transforms information.

---

53. Vision Transformers for Classification

ViTs initially outperformed:

- classic CNNs.

But modern CNNs evolved.

---

54. ConvNeXt

Modernized CNN architecture.

Borrowed:

- transformer design principles.

Achieved competitive classification accuracy.

---

55. Main Difference in 2025

Classification:

- ViTs and CNNs nearly tied.

Generation:

- transformers dominate.

---

56. Diffusion Transformers (DiT)

Transformers used as:

- diffusion model backbones.

Replace:

- U-Net CNNs.

---

57. Why Transformers Win in Generation

Self-attention enables:

- global coherence
- high-resolution structure
- better scaling.

---

58. FID Score

Fréchet Inception Distance.

Measures:

- similarity between generated and real image distributions.

Lower FID:

- better generation quality.

---

59. Precision vs Recall in Generation

Precision

Measures realism.

Are generated images convincing?

Recall

Measures diversity.

Does generated set cover real data space?

---

60. Precision-Recall Tradeoff

High precision + low recall:

- realistic but repetitive outputs.

High recall + low precision:

- diverse but low-quality outputs.

---

61. Scaling Laws

Transformers improve predictably with:

- larger model size
- more compute.

---

62. Key Scaling Insight

Increasing model size:

- more effective than simply increasing sampling steps.

---

63. Why Scaling Laws Matter

Predictable improvement:

- motivates industry investment.

Transformers scale smoothly.

---

64. Why Industry Pivoted to Transformers

Reasons:

- predictable scaling

- multimodal compatibility
- strong generation performance
- global attention.

---

65. Most Important Conceptual Insight

Vision Transformers reframe images as:

Structured token sequences.

---

66. Important Equations Cheat Sheet

Number of Patches

$N = (H/P)(W/P)$

Positional Encoding

$X' = X + P$

Attention Equation

$\mathrm{softmax}(QK^T / \sqrt{d\_h})V$

Residual Connection

$x + y$

---

67. Frequently Asked Open-Book Exam Questions

Q1. Why divide images into patches?

Answer Points:

- reduce sequence length
- make self-attention computationally feasible.

Q2. Why are positional encodings necessary?

Answer Points:

- transformers ignore order naturally
- positional information restores spatial geometry.

Q3. Difference between absolute and relative positional encodings?

Answer Points:

- absolute = fixed coordinates
- relative = spatial relationships.

Q4. Why is RoPE useful?

Answer Points:

- better extrapolation to unseen resolutions.

Q5. What is the role of the CLS token?

Answer Points:

- global information aggregator
- classification summary token.

Q6. Why divide by $\sqrt{d\_h}$ in attention?

Answer Points:

- prevents softmax saturation
- stabilizes gradients.

Q7. Difference between attention and FFN?

Answer Points:

- attention exchanges information
- FFN performs nonlinear reasoning.

Q8. Why do transformers outperform CNNs in generation?

Answer Points:

- global attention
- better scaling
- stronger long-range consistency.

Vision Language Models (VLMs) —

1. Core Idea of Vision Language Models (VLMs)

Vision Language Models learn:

- a shared embedding space
- between images and language.

Goal:

- connect visual semantics with textual semantics.

---

2. Historical Shift in Computer Vision

Old computer vision:

- fixed classification labels.

Example:

- cat
- dog
- airplane.

Model could only recognize:

- predefined categories.

---

3. Open Vocabulary Recognition

VLMs replaced:

- fixed labels

with:

- language supervision.

Allows:

- zero-shot recognition
- open-ended understanding.

---

4. Two Main VLM Families

(a) Dual Encoder Models

Example:

- CLIP.

(b) Fusion / Encoder-Decoder Models

Example:

- BLIP
- Flamingo
- LLaVA.

---

5. Dual Encoder Architecture

Two separate encoders:

Image Encoder

Processes images.

Text Encoder

Processes text.

Both remain:

- isolated during encoding.

---

6. Core Property of Dual Encoders

Image encoder never sees text.

Text encoder never sees image.

Only interaction:

- final embedding space.

---

7. Fusion Model Architecture

Visual tokens and text tokens:

- interact directly.

Uses:

- cross-attention.

Enables:

- reasoning
- generation
- captioning.

---

8. Retrieval vs Generation

Dual Encoders

Best for:

- retrieval
- search
- zero-shot classification.

Fusion Models

Best for:

- captioning
- VQA
- reasoning
- dialogue.

---

9. CLIP Architecture

CLIP contains:

- image encoder $f\theta(x)$
- text encoder $g\phi(y)$.

---

10. Embedding Vectors

Both encoders output:

512-dimensional vectors.

These vectors live in:

- shared embedding space.

---

11. L2 Normalization

Critical concept.

All embeddings normalized to:

unit length.

Meaning:

$||z||_2 = 1$

---

12. Why Normalize?

Removes:

- vector magnitude.

Preserves only:

- angular direction.

---

13. Hypersphere Geometry

After normalization:

- all embeddings lie on surface of hypersphere.

Similarity depends only on:

- angle between vectors.

---

14. Cosine Similarity

Similarity between image and text:

$\cos(\theta)$

Equivalent to:

- dot product of normalized vectors.

---

15. Interpretation of Cosine Similarity

Similarity = 1

Vectors aligned perfectly.

Similarity = 0

Vectors orthogonal.

No semantic relation.

Similarity = −1

Vectors opposite.

---

16. Contrastive Pretraining

Core CLIP training method.

Goal:

- pull matching pairs together.
- push mismatched pairs apart.

---

17. Batch Similarity Matrix

For batch size B:

Compute:

B × B similarity matrix.

Rows:

- images.

Columns:

- captions.

---

18. Positive Pairs

True image-caption matches.

Located on:

- diagonal of matrix.

---

19. Negative Pairs

All off-diagonal entries.

Represent:

- mismatched image-text pairs.

---

20. Symmetric Cross Entropy Loss

CLIP optimizes:

Image → Text retrieval

AND

Text → Image retrieval.

Simultaneously.

---

21. Temperature Parameter τ

Very important exam topic.

Similarity logits divided by:

τ

before softmax.

---

22. Why Temperature Scaling is Needed

Without scaling:

- softmax distributions too flat.
- weak gradients.
- slow learning.

---

23. Small Temperature Effect

Small τ:

- amplifies logits.
- creates sharp probability distribution.

Focuses learning on:

- hard negatives.

---

24. Hard Negatives

Mismatched pairs that:

- still look semantically similar.

Example:

- Burj Khalifa vs Burj Al Arab.
- cat vs dog.

---

25. Gradient Geometry

Contrastive gradients perform:

Pulling Force

Move matching embeddings together.

Repulsive Force

Push confusing negatives apart.

26. MNIST Example

Image of sloppy “7” may resemble “3”.

Contrastive loss:

- pulls 7 toward text “7”
- pushes away from text “3”.

27. Zero-Shot Classification

Major CLIP capability.

No task-specific classifier required.

28. Prompt Engineering

Instead of label:

"dog"

Use:

"a photo of a dog"

29. Why Prompt Engineering Helps

Aligns input with:

- natural language structure seen during training.

30. Prompt Ensembling

Use many prompt templates.

Example:

- a photo of a dog
- blurry dog photo
- black-and-white dog photo.

Average embeddings.

31. Why Prompt Ensembling Works

Reduces:

- prompt-specific noise.

Improves:

- robustness.

32. t-SNE Visualization

t-SNE reduces:

- high-dimensional embeddings
- into 2D visualization.

---

33. Meaning of t-SNE Clusters

Images and captions for same concept:

- cluster together.

Demonstrates:

- successful semantic alignment.

---

34. Geometric View of Classification

Classification becomes:

- nearest-neighbor search in embedding space.

---

35. Cross-Modal Retrieval

Given text:

- retrieve nearest image embeddings.

Given image:

- retrieve nearest captions.

---

36. Histograms of Similarity

Correct image-text pairs:

- higher cosine similarity.

Incorrect pairs:

- lower similarity.

---

37. Limitation of Dual Encoders

Dual encoders cannot:

- generate text.
- reason sequentially.

They only:

- evaluate similarity.

---

38. Critical Exam Trap

CLIP is NOT generative.

It does NOT generate images.

---

39. CLIP's Real Role

CLIP acts as:

- evaluator
- scorer
- alignment model.

---

40. Diffusion Models Generate Images

Diffusion models:

- synthesize pixels.

CLIP:

- guides diffusion process.

---

41. CLIP Guidance Intuition

CLIP scores intermediate noisy images.

Diffusion model adjusts generation toward:

- higher CLIP similarity.

---

42. Fusion Models

Fusion models enable:

- image captioning
- visual QA
- reasoning.

---

43. Visual Memory

Image encoder outputs:

- sequence of visual tokens.

These tokens form:

- visual memory bank.

---

44. Cross-Attention Routing

Queries

Come from text decoder.

Keys and Values

Come from visual memory.

---

45. Cross-Attention Intuition

Text asks:

"What visual information do I need next?"

Image provides:

- relevant visual evidence.

---

46. Autoregressive Captioning

Decoder generates:

- one token at a time.

Conditioned on:

- previous text
- visual memory.

---

47. Distillation

Used to compress giant VLMs.

Goal:

- train small student model
- to imitate large teacher model.

---

48. Embedding Regression Loss

Student embedding forced to match:

- teacher embedding.

Uses:

- mean squared error.

---

49. Why MSE Alone is Insufficient

MSE matches:

- individual vectors.

But ignores:

- relationships between concepts.

---

50. Relational Knowledge

Teacher model contains:

- semantic relationships.

Example:

- dog slightly similar to cat.

- airplane unrelated to dog.

---

51. KL Divergence Distillation

KL divergence transfers:

- probability distributions.

Preserves:

- relational geometry.

---

52. Dark Knowledge

Soft probabilities encode:

- nuanced semantic structure.

Example:

- dog somewhat related to deer.

---

53. Why Dark Knowledge Matters

Student learns:

- structure of embedding space.

Not just:

- hard labels.

---

54. Major VLM Failure Modes

(a) Bias and Spurious Correlations

(b) Weak Spatial Grounding

(c) Hallucination.

---

55. Spurious Correlations

Model learns shortcuts.

Example:

- cows often appear in grass.

Model may associate:

- grass → cow.

---

56. Weak Spatial Grounding

Global pooling destroys:

- fine spatial relationships.

Model struggles with:

- attribute binding.

---

57. Attribute Binding Problem

Model confuses:

- red cup on blue book
  vs
- blue cup on red book.

---

58. Hallucination

Fusion models may generate:

- plausible but false details.

Reason:

- language priors overpower visual evidence.

---

59. SigLIP

Alternative to CLIP.

Replaces:

- global softmax

with:

- pairwise sigmoid objective.

---

60. Why SigLIP Helps

Advantages:

- memory efficient
- scalable training
- no giant global competition matrix.

---

61. Dense Prediction Extensions

Models like:

- CLIPSeg

Use intermediate patch features.

Enable:

- segmentation
- localization.

---

62. Open Vocabulary Detection

Models:

- OWL-ViT
- Grounding DINO.

Combine:

- CLIP semantics
- object detection.

---

63. VLM + LLM Fusion

Models:

- BLIP-2
- LLaVA.

Bridge:

- visual tokens
- large language models.

---

64. Most Important Conceptual Insight

VLMs transform:

- vision
- language

into:

- shared geometric embedding spaces.

---

65. Important Equations Cheat Sheet

Cosine Similarity

sim(x,y) = x·y

(for normalized vectors)

Contrastive Objective

Pull positives together.

Push negatives apart.

Temperature Scaling

logits / τ

Symmetric Retrieval Loss

Image→Text + Text→Image.

---

66. Frequently Asked Open-Book Exam Questions

Q1. Difference between dual encoders and fusion models?

Answer Points:

- dual encoders separate modalities
- fusion models combine modalities early.

Q2. Why normalize embeddings?

Answer Points:

- remove magnitude
- compare only angular similarity.

Q3. Why use temperature scaling?

Answer Points:

- sharpen probability distributions
- focus on hard negatives.

Q4. What are hard negatives?

Answer Points:

- semantically similar but incorrect pairs.

Q5. Why is CLIP not generative?

Answer Points:

- outputs embeddings only
- does not synthesize pixels.

Q6. Explain cross-attention routing.

Answer Points:

- text decoder generates queries
- image provides keys/values.

Q7. Why use KL divergence in distillation?

Answer Points:

- transfer relational geometry
- preserve semantic distributions.

Q8. Explain hallucination in VLMs.

Answer Points:

- language priors dominate visual evidence.

Training and Adapting Vision Language Models (VLMs) —

1. Core Problem of VLM Adaptation

Pretrained VLMs are general-purpose.

Problem:

- domain-specific tasks require adaptation.

Example:

- distinguishing visually similar lizard species.

---

2. Budget Ladder of Adaptation

Adaptation methods increase in:

- compute cost
- data requirements
- complexity.

Main stages:

1. Zero-shot
2. Few-shot / In-context learning
3. Retrieval-Augmented Generation (RAG)
4. Supervised Fine-Tuning (SFT)
5. Direct Preference Optimization (DPO)
6. Online Reinforcement Learning (RLOO/RLHF)

---

3. Zero-Shot Inference

No training.

No parameter updates.

Uses:

- frozen pretrained weights.

---

4. Conditional Probability in VLMs

Core inference equation:

$P(y \mid x, v)$

where:

- x = text prompt

- v = image
- y = generated response.

---

5. Autoregressive Factorization

Model predicts:

- one token at a time.

Factorization:

$P(y_t \mid y_{<t}, x, v)$

---

6. Meaning of Autoregressive Generation

Current token prediction depends on:

- previous generated tokens
- prompt
- image.

---

7. Why Zero-Shot Often Fails

Model lacks:

- domain-specific knowledge.

Example:

- cannot distinguish rare reptile morphology.

---

8. Few-Shot / In-Context Learning

No parameter updates.

Still uses:

- frozen θ.

Adaptation happens through:

- context engineering.

---

9. Why Few-Shot Works

Transformers use:

- attention mechanisms.

Examples modify:

- hidden state trajectories.

---

10. Attention Intuition

Examples act as:

- temporary memory files.

Model retrieves:

- formatting patterns
- task structure
- label constraints.

---

11. Three Functions of Few-Shot Examples

(a) Format Induction

Teaches output structure.

(b) Task Disambiguation

Clarifies desired task.

(c) Label Space Restriction

Narrows candidate outputs.

---

12. Recency Bias

Transformers attend more strongly to:

- nearby examples.

Example order matters.

---

13. Limitation of Few-Shot Learning

Cannot inject:

- new factual knowledge.

Only manipulates:

- context.

---

14. Retrieval-Augmented Generation (RAG)

Adds:

- external searchable memory.

---

15. RAG Goal

Combine:

- retrieval
- generation.

---

16. RAG Probability Equation

P(y|x,v)

Σ P(z|q(x,v)) P(y|x,v,z)

---

17. Meaning of Variables

z

Retrieved document.

---

q(x,v)

Joint query representation.

P(z|q)

Retriever probability.

P(y|x,v,z)

Generator probability.

---

18. RAG Intuition

Retriever selects:

- useful documents.

Generator uses:

- retrieved context.

Final answer combines:

- weighted contributions.

---

19. Posterior Responsibility Weight

Very important concept.

Measures:

- how responsible a retrieved document was for generating correct answer.

---

20. Why Posterior Responsibility Matters

Provides learning signal to:

- retriever.

Helpful documents:

- receive higher weight.

---

21. Main RAG Failure Mode

If retriever fetches:

- wrong documents

Generator hallucinates confidently.

---

22. Supervised Fine-Tuning (SFT)

First method that:

- changes model weights.

Updates:

- θ directly.

---

23. SFT Dataset Structure

Training triples:

(image, prompt, correct answer)

---

24. SFT Objective

Use:

Negative Log Likelihood (NLL).

---

25. SFT Loss Formula

L_SFT(θ)

$-(1/N) \Sigma \Sigma m_{i,t} \log P(y_{i,t} \mid y_{i,<t}, x_i, v_i)$

---

26. Meaning of SFT Loss

Model punished when:

- assigning low probability to correct token.

---

27. Loss Mask

Most important debugging concept.

Binary mask:

- decides which tokens contribute to loss.

---

28. Why Loss Mask is Needed

We do NOT want model to learn:

- user prompts.

We ONLY train on:

- assistant outputs.

---

29. Mask Values

m = 0

Ignore token.

---

m = 1

Include token in loss.

---

30. Critical Qwen Debugging Issue

If assistant mask becomes:

all zeros

then:

- loss = 0
- gradients = 0
- model learns nothing.

---

31. SFT Advantages

Allows:

- genuine new knowledge injection.

---

32. SFT Risks

(a) Mode Collapse

Model becomes rigid.

(b) Catastrophic Forgetting

Model forgets previous abilities.

---

33. Mode Collapse

Model overfits output format.

Example:

- only outputs rigid JSON.

---

34. Catastrophic Forgetting

Overwrites:

- previous semantic capabilities.

Example:

- becomes expert on reptiles

- loses general reasoning.

---

35. LoRA (Low-Rank Adaptation)

Efficient alternative to full SFT.

Base weights:

- frozen.

Train only:

- small adapter matrices.

---

36. LoRA Equation

$\Delta W = AB$

where:

- A and B are low-rank matrices.

---

37. Why LoRA Saves Memory

Instead of updating:

- full matrix.

Only update:

- tiny low-rank adapters.

---

38. LoRA Rank r

Controls:

- adaptation capacity.

Small r:

- efficient but weak.

Large r:

- expressive but risks overfitting.

---

39. QLoRA

Quantized LoRA.

Frozen base model stored in:

- 4-bit precision.

Adapters remain:

- high precision.

---

40. Why QLoRA Works

Compresses:

- huge base model.

Preserves learning in:

- adapters.

---

41. Attention Adapters

Applied to:

- Q projections
- K projections
- V projections
- output projections.

---

42. What Attention Adapters Learn

Teach model:

- where to look.

---

43. MLP Adapters

Applied inside:

- feedforward layers.

---

44. What MLP Adapters Learn

Teach model:

- what visual patterns mean.

---

45. Why Both Attention + MLP Matter

Attention:

- localization.

MLP:

- semantic interpretation.

Need both for strong adaptation.

---

46. Overfitting in LoRA

High rank may learn:

- background shortcuts.

Example:

- red rock → lizard species.

Instead of:

- actual morphology.

---

47. Direct Preference Optimization (DPO)

Goal:

- teach nuanced preferences.

Instead of hard labels.

---

48. DPO Dataset

Each example contains:

- prompt
- image
- preferred answer y_w
- rejected answer y_l.

---

49. DPO Core Idea

Increase probability of:

- preferred answer.

Decrease probability of:

- rejected answer.

---

50. DPO Loss

Uses:

- logistic preference objective.
- KL regularization.

---

51. Reference Model in DPO

Frozen model acts as:

- stability anchor.

Prevents:

- reward hacking
- language collapse.

---

52. Role of β in DPO

Controls:

- strength of deviation from reference model.

Large β:

- tight leash.

Small β:

- more freedom.

---

53. Why DPO Was Revolutionary

Avoids:

- explicit reward model.
- complex RL pipelines.

Much simpler than RLHF.

---

54. Limitation of DPO

Still offline.

Model only learns from:

- static examples.

Cannot explore new behaviors.

---

55. Online Reinforcement Learning

Allows:

- active exploration.
- live response generation.

---

56. RLHF Pipeline

1. SFT policy
2. Reward model
3. Online generation
4. Reward scoring
5. RL optimization.

---

57. Reward Model

Separate neural network.

Outputs:

- scalar quality score.

---

58. RLOO (Reinforce Leave-One-Out)

Efficient RL algorithm.

Used in:

- Qwen training ecosystem.

---

59. RLOO Workflow

Generate:

- multiple completions.

Compare:

- each completion against average of others.

---

60. Advantage Estimate

$A_i = R_i - B_i$

where:

- R_i = reward
- B_i = baseline.

---

61. Positive Advantage

Completion better than average.

Increase probability.

---

62. Negative Advantage

Completion worse than average.

Decrease probability.

---

63. KL Penalty in RL

Prevents:

- drifting too far from reference policy.

Maintains:

- linguistic sanity.

---

64. Why Online RL is Powerful

Model explores:

- unseen strategies.
- novel reasoning paths.

---

65. Why Online RL is Expensive

Requires:

- active model

- frozen reference model
- reward model
- rollout storage.

Huge VRAM usage.

---

66. Most Important Conceptual Insight

Each rung of adaptation changes:

- where learning happens.

---

67. Adaptation Ladder Summary

Zero-shot

No learning.

Few-shot

Context adaptation.

RAG

External memory.

SFT

Weight updates.

DPO

Preference shaping.

RL

Behavior optimization through exploration.

---

68. Important Equations Cheat Sheet

Conditional Probability

$P(y|x,v)$

Autoregressive Factorization

$P(y_t \mid y_{<t}, x, v)$

RAG Mixture

$\Sigma P(z|q) P(y|x,v,z)$

SFT Loss

$-\log P(\text{correct token})$

LoRA

$\Delta W = AB$

Advantage Estimate

$A_i = R_i - B_i$

69. Frequently Asked Open-Book Exam Questions

Q1. Why does few-shot learning work without updating weights?

Answer Points:

- modifies hidden-state context
- changes attention retrieval patterns.

Q2. Why is RAG useful?

Answer Points:

- injects external knowledge
- avoids retraining model.

Q3. Why use a loss mask in SFT?

Answer Points:

- avoid training on prompts
- train only assistant outputs.

Q4. Why is LoRA memory efficient?

Answer Points:

- updates only low-rank matrices
- freezes base model.

Q5. Difference between attention adapters and MLP adapters?

Answer Points:

- attention = where to look
- MLP = what it means.

Q6. Why is DPO simpler than RLHF?

Answer Points:

- no explicit reward model optimization loop.

Q7. What does the KL penalty do?

Answer Points:

- prevents reward hacking
- keeps model close to baseline behavior.

Q8. Why is RL more powerful than DPO?

Answer Points:

- supports exploration
- generates novel behaviors.

Neural Network Quantization —

1. Core Idea of Quantization

Quantization compresses:

- high-precision neural network numbers

into:

- low-precision discrete representations.

Goal:

- reduce memory
- increase speed
- reduce power consumption.

---

2. Why Quantization is Necessary

Modern LLMs contain:

- billions of parameters.

Example:

- 175B parameter model.

Using FP32:

- requires hundreds of GB of memory.

---

3. Three Hardware Bottlenecks

(a) Memory Footprint

Can model fit in RAM/VRAM?

(b) Memory Bandwidth

How fast can data move?

(c) Arithmetic Throughput

How many operations per second can hardware execute?

---

4. Floating Point Precision

Standard models use:

FP32 (32-bit float)

Provides:

- continuous high precision.

---

5. Quantization Goal

Replace:

- FP32 numbers

with:

- INT8
- INT4
- binary values.

---

6. Fundamental Tradeoff

Lower Precision

- faster
- smaller
- energy efficient.

Higher Precision

- more accurate
- less approximation error.

---

7. Uniform Affine Quantization

Core quantization method.

Process:

1. Divide by scale

2. Round to nearest integer
3. Clip into valid integer range.

---

8. Scale Parameter

Scale determines:

- spacing between quantization levels.

Acts like:

- staircase step size.

---

9. Resolution vs Range Tradeoff

Small Scale

High precision.

But:

- limited numerical range.

Large Scale

Large range.

But:

- poor resolution.

---

10. Clipping

If value exceeds integer range:

- value saturates.

Example:

- clipped to ±127 for INT8.

---

11. Outliers

Large rare activations.

Outliers force:

- huge quantization scales.

Destroy precision for normal values.

---

12. Calibration

Engineers run calibration data through model.

Purpose:

- estimate activation ranges.

---

13. Min-Max Calibration

Uses:

- absolute minimum
- absolute maximum.

Problem:

- highly sensitive to outliers.

---

14. Percentile Calibration

Uses:

- statistical percentiles.

Ignores extreme outliers.

Preserves:

- normal value precision.

---

15. Per-Tensor Quantization

Single scale factor for:

- entire tensor.

---

16. Problem with Per-Tensor Quantization

Different channels may have:

- vastly different distributions.

One global scale destroys small channels.

---

17. Per-Channel Quantization

Each channel gets:

- independent scale.

Preserves:

- local precision.

---

18. Group-Wise Quantization

Intermediate strategy.

Small groups share:

- local scales.

Balances:

- efficiency
- accuracy.

---

19. Why Quantization Errors Matter

Neural networks are:

- recursive dynamical systems.

Small errors:

- compound across layers.

---

20. Butterfly Effect in Quantization

Tiny rounding errors:
→ propagate through network
→ amplify over depth.

---

21. Loss Landscape

Neural network training creates:

- valleys and cliffs in parameter space.

Quantization perturbs weights:

- away from optimum.

---

22. Taylor Expansion View

Approximate loss increase caused by:

- quantization perturbation.

---

23. Gradient Term

First-order term.

At trained optimum:

- gradient ≈ 0.

---

24. Hessian Matrix

Second-order curvature matrix.

Measures:

- sensitivity to perturbations.

---

25. Flat vs Sharp Directions

Flat Direction

Quantization causes little damage.

Sharp Direction

Tiny perturbation causes huge loss increase.

---

26. Key Insight

Not all quantization errors are equally harmful.

Curvature matters.

---

27. Quantization-Aware Training (QAT)

Model trained while simulating:

- quantization noise.

---

28. QAT Goal

Teach model to:

- survive quantization during learning.

---

29. Fake Quantizers

Inserted during:

- forward pass.

Model experiences:

- quantized activations.

---

30. Core Mathematical Problem

Quantization functions are:

- non-differentiable.

Backpropagation should fail.

---

31. Straight-Through Estimator (STE)

Most important QAT concept.

---

32. STE Intuition

Forward Pass

Use real quantization.

Backward Pass

Pretend quantizer is identity function.

Gradient passes straight through.

---

33. Why STE Works

Provides:

- useful surrogate gradient.

Allows optimization despite:

- non-differentiability.

---

34. PACT (Parameterized Clipping Activation)

Makes clipping threshold:

- learnable parameter.

---

35. Why PACT Helps

Model learns:

- optimal clipping tradeoff automatically.

---

36. LSQ (Learned Step Size Quantization)

Makes quantization step size:

- trainable.

---

37. LSQ Intuition

Model dynamically adjusts:

- spacing of quantization grid.

---

38. Binary Quantization

1-bit quantization.

Weights become:

- +1
- −1.

---

39. Ternary Quantization

2-bit style quantization.

Weights become:

- +1
- 0
- −1.

---

40. Why Ternary is Better

Zero enables:

- sparsity.

- disabling unimportant connections.

---

41. Post-Training Quantization (PTQ)

Compress already-trained frozen models.

No retraining.

---

42. Round-To-Nearest (RTN)

Naive PTQ baseline.

Simply rounds:

- each weight independently.

---

43. Why RTN Fails

Ignores:

- network structure
- interactions
- curvature.

---

44. AdaRound

Adaptive rounding method.

Optimizes:

- rounding decisions collaboratively.

---

45. AdaRound Key Insight

Sometimes mathematically wrong local rounding:
produces better global output.

---

46. GPTQ

Curvature-aware PTQ method.

Uses:

- inverse Hessian approximation.

---

47. GPTQ Intuition

Quantize one column.

Then compensate remaining weights:

- to cancel induced error.

---

48. Why GPTQ Works Well

Accounts for:

- curvature sensitivity.

Maintains:

- model quality at low bits.

---

49. W8A8 Problem

Large LLMs develop:

- extreme activation outliers.

INT8 activations become unstable.

---

50. SmoothQuant

Shifts difficulty from:

- activations

into:

- weights.

---

51. SmoothQuant Equation Idea

Scale activations down.

Scale weights up proportionally.

Overall computation remains unchanged.

---

52. Why SmoothQuant Helps

Activations become:

- easier to quantize.

Weights absorb:

- dynamic range.

---

53. AWQ (Activation-Aware Weight Quantization)

Identifies:

- most important weights.

Protects them during quantization.

---

54. Salient Weights

Small subset of weights carries:

- majority of semantic importance.

---

55. AWQ Strategy

Scale salient weights up.

Give them:

- higher effective precision.

---

56. Floating-Point Islands

Nonlinear functions often require:

- dequantization back to FP32.

Examples:

- softmax
- GELU
- LayerNorm.

---

57. Problem with Floating-Point Islands

Repeated:

- INT → FP32 → INT conversion.

Destroys:

- efficiency gains.

---

58. I-BERT

Integer-only transformer execution.

---

59. How I-BERT Works

Approximates nonlinear operations using:

- integer-friendly polynomial approximations.

---

60. Why I-BERT Matters

Enables:

- full integer inference pipeline.

No floating-point islands.

---

61. Quantization in Generative Models

Much harder than:

- classification models.

---

62. Why Generative Models Are Sensitive

Generation is:

- iterative.

Errors compound across steps.

---

63. Euler Integration Drift

Tiny quantization errors:
→ accumulate across iterative denoising.

Trajectory drifts off manifold.

---

64. Example of Drift

Image generation artifacts:

- distorted hands
- incorrect anatomy.

May originate from:

- compounded quantization errors.

---

65. Time-Aware Quantization

Future methods optimize:

- entire generation trajectory.

Not just:

- single forward pass.

---

66. Most Important Conceptual Insight

Quantization is NOT just compression.

It is:

- geometric approximation under hardware constraints.

---

67. Important Equations Cheat Sheet

Uniform Affine Quantization

q = round(x / s)

Scale Tradeoff

Small s → precision

Large s → range

Loosely Approximate Taylor Expansion

Loss increase depends on:

- Hessian curvature.

SmoothQuant Idea

$(W/s)(sX) = WX$

---

68. Frequently Asked Open-Book Exam Questions

Q1. Why is quantization necessary?

Answer Points:

- reduce memory
- improve throughput
- lower energy cost.

Q2. Explain resolution vs range tradeoff.

Answer Points:

- small scale improves precision
- large scale prevents clipping.

Q3. Why are outliers dangerous?

Answer Points:

- force huge scales
- destroy precision for normal values.

Q4. Why does Hessian curvature matter?

Answer Points:

- quantization damage depends on local sensitivity.

Q5. Why does STE work?

Answer Points:

- provides surrogate gradients
- ignores staircase discontinuities.

Q6. Difference between QAT and PTQ?

Answer Points:

- QAT retrains with quantization noise
- PTQ compresses frozen models.

Q7. Why is GPTQ superior to RTN?

Answer Points:

- curvature-aware compensation.

Q8. Why are generative models harder to quantize?

Answer Points:

- iterative errors compound over time.